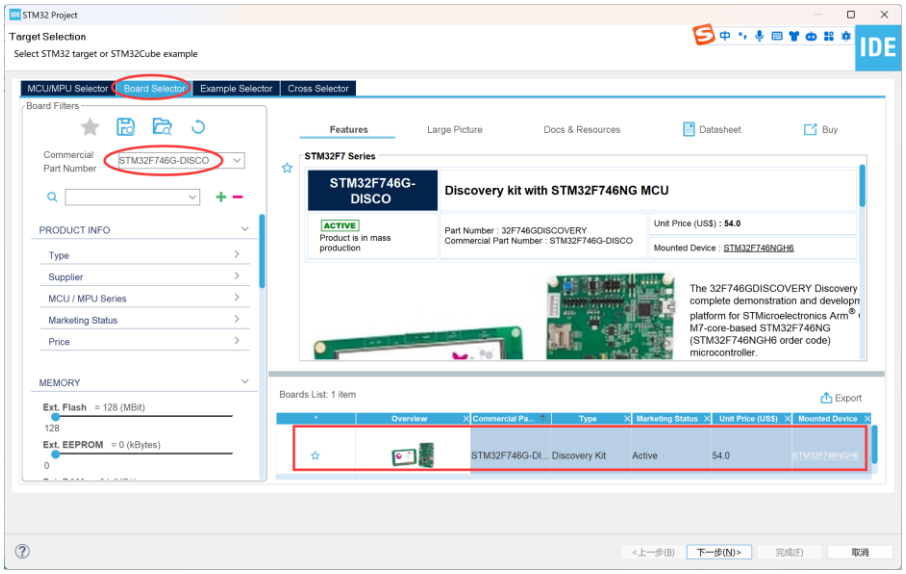
实践三 ARM 汇编编程实践

一、ASM 汇编项目生成

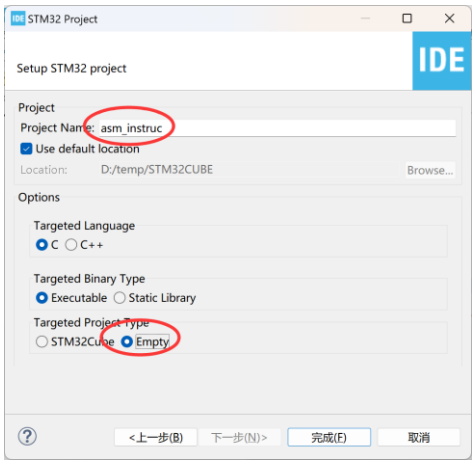
(1) 启动 STM32CubeIDE：双击桌面上的 STM32CubeIDE 图标启动开发环境。

(2) 创建新项目：点击菜单栏中的"File" -> "New" -> "STM32 Project"，弹出项目创建向导。

(3) 选择开发板：在向导中选择合适的开发板（STM32F746G-DISCO）。



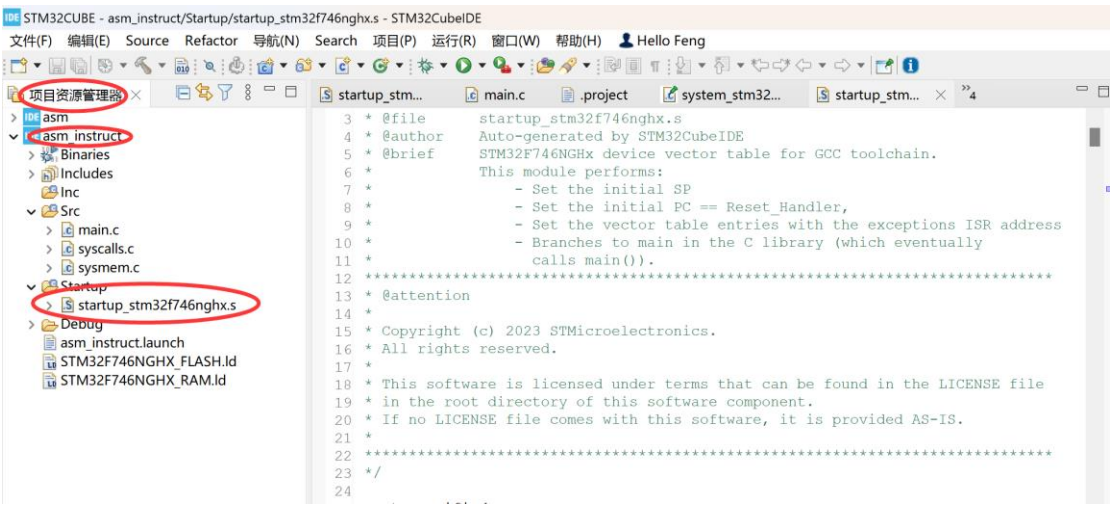
(4) 设置项目属性：项目类型选择为“Empty”。



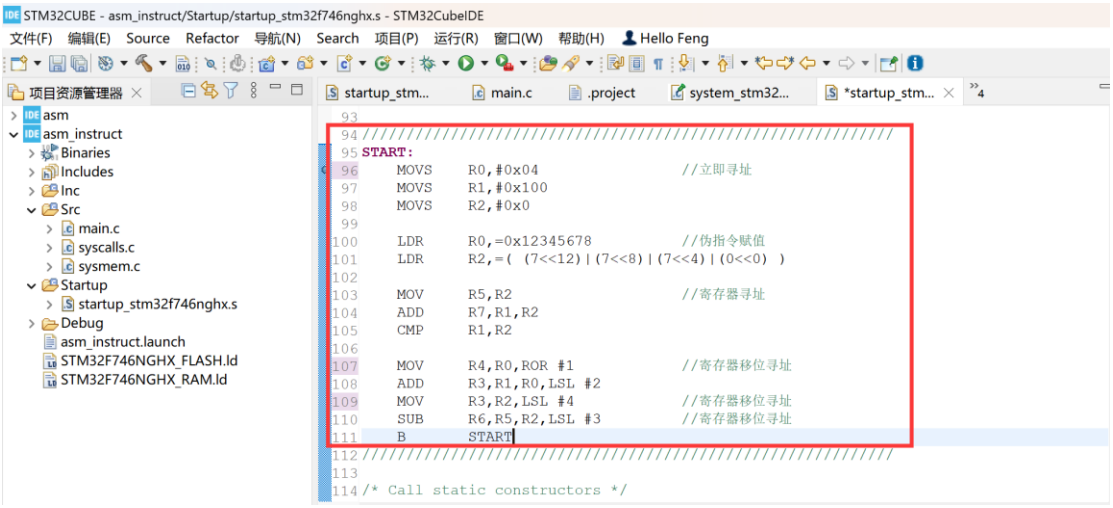
(5) 完成项目创建：点击"Finish"按钮完成项目创建。

二、编写代码

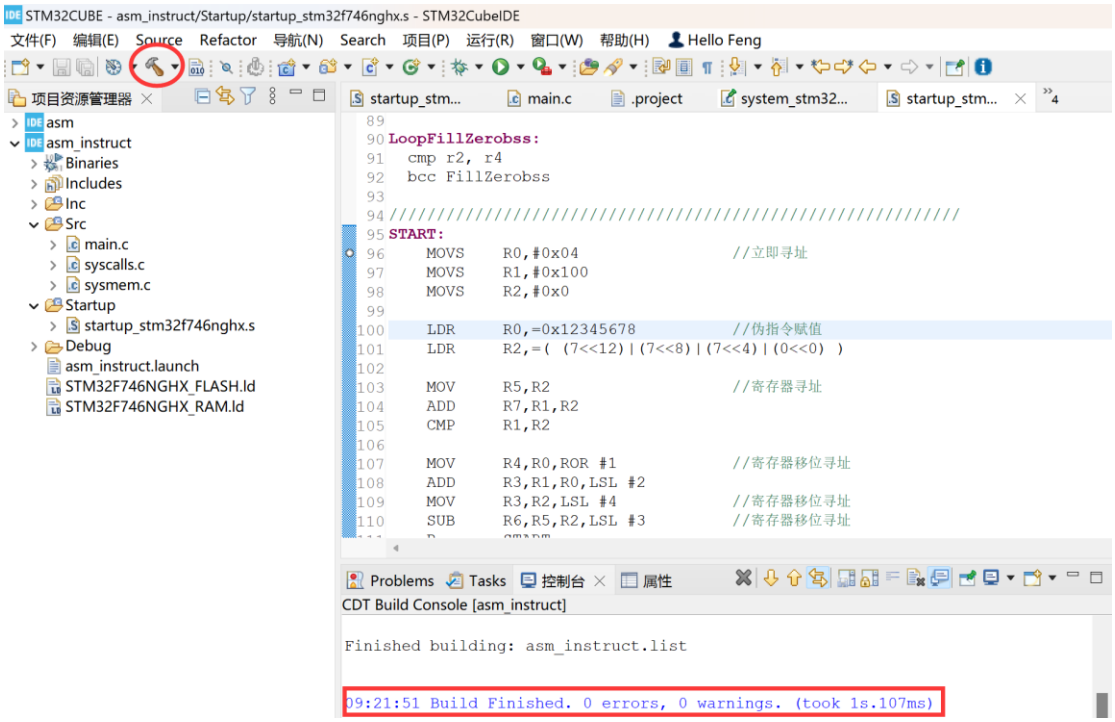
(1) 打开启动汇编文件：在项目资源管理器中找到“asm_instruct”项目，在其“Startup”文件下面，双击“startup_stm32f746nghx.s”打开该文件”。



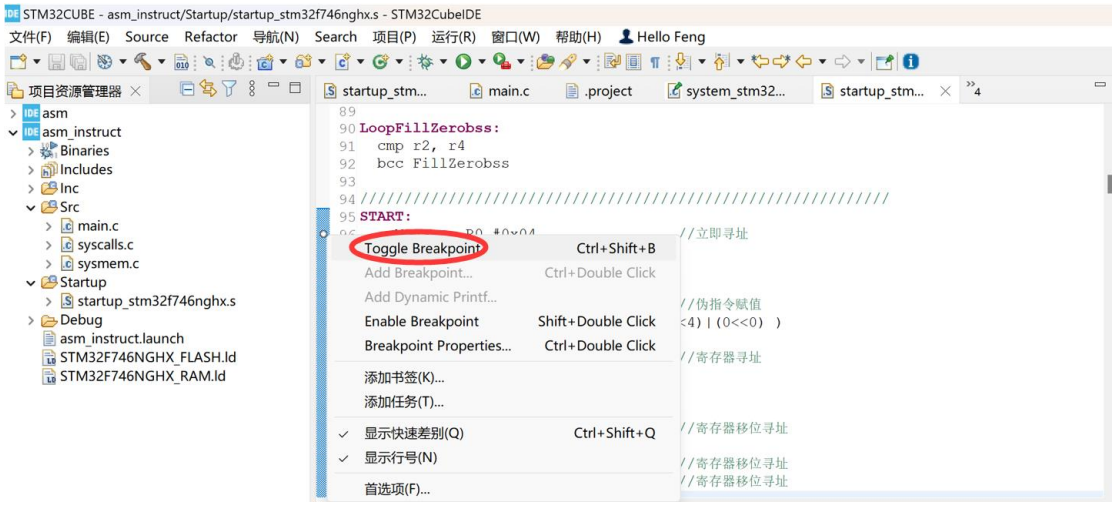
(2) 编辑汇编文件：在“startup_stm32f746nghx.s”文件的 94 行开始，添加自己的汇编代码。



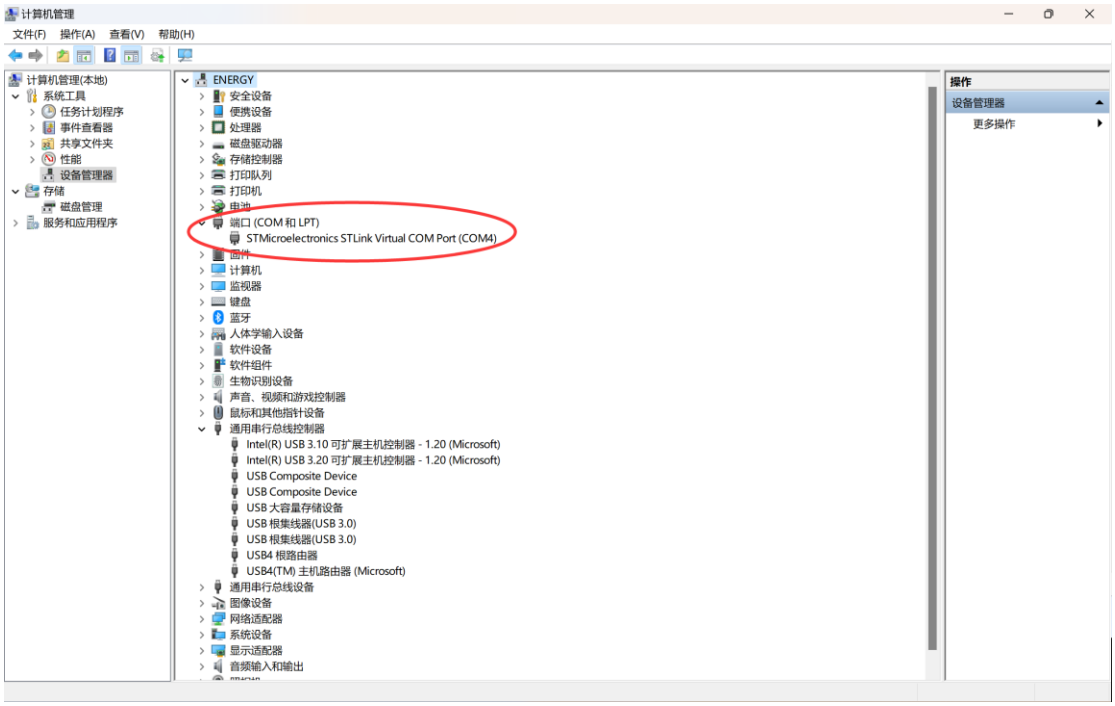
(3) Build 项目，确认编译无错误。



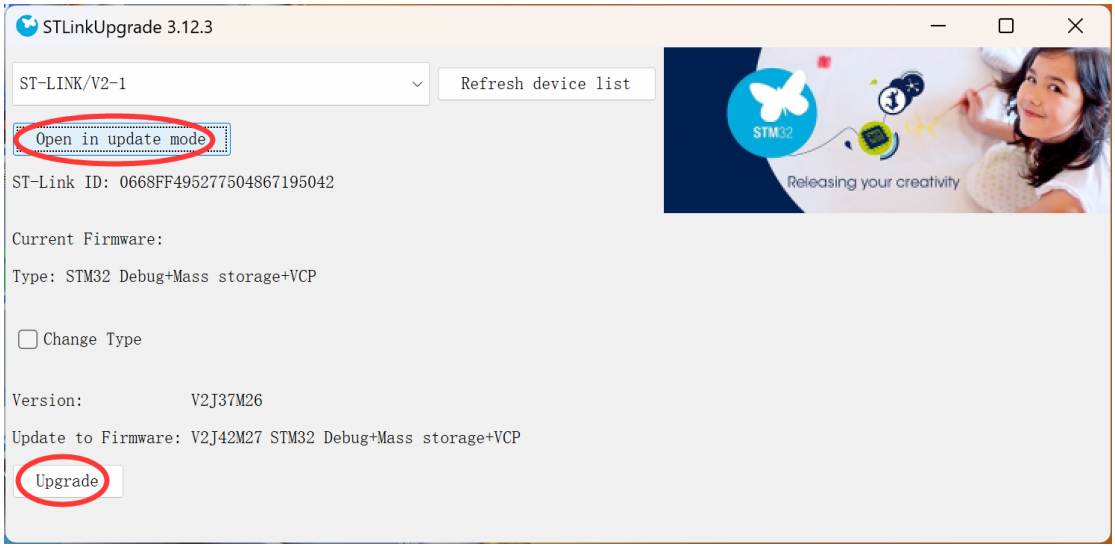
(4) 打断点：在“startup_stm32f746nghx.s”文件 96 行点鼠标右键，添加一个断点。



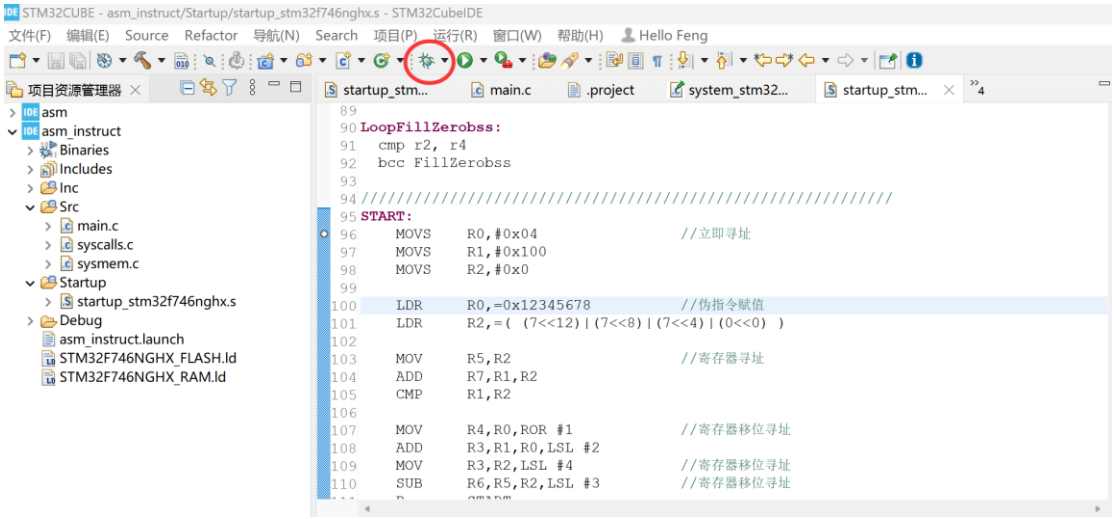
(5) 插上开发板连接线，在 windows 设备管理器中查看开发板识别是否正常。



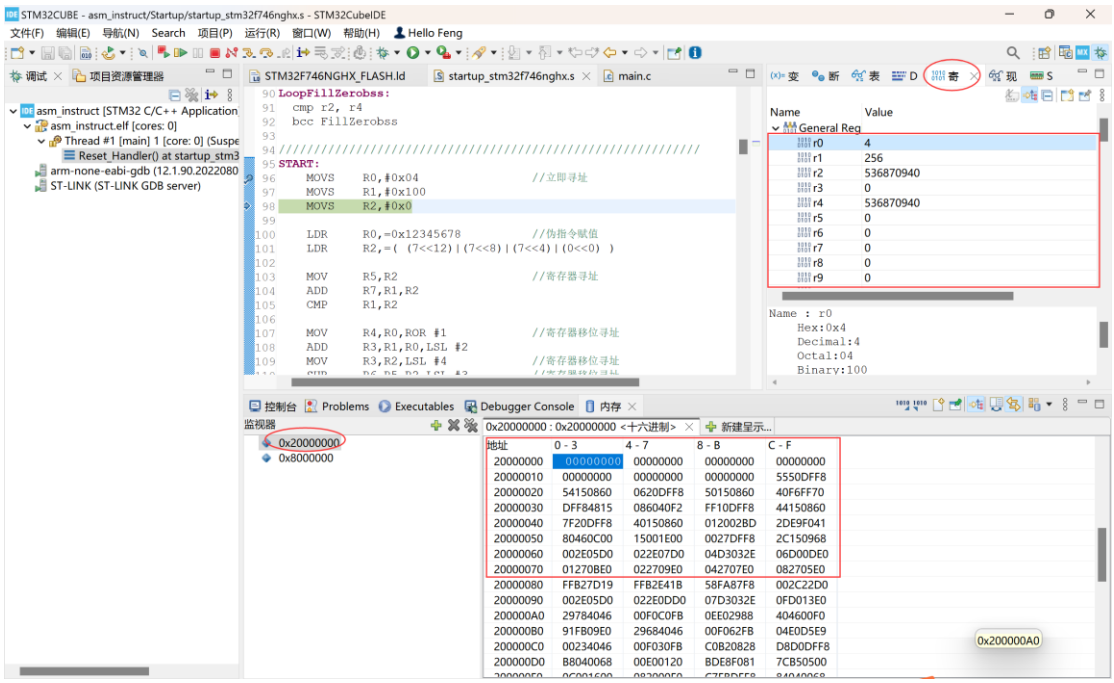
(6) 若有必要，更新开发板 Firmware。



(7) 点击调试按钮，进入调试状态。



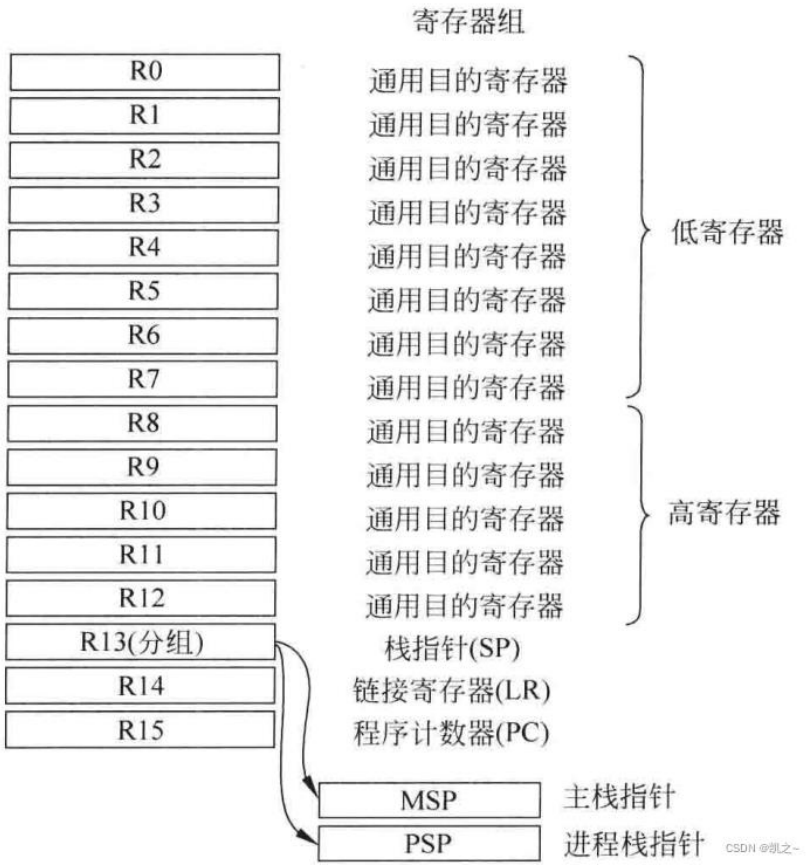
(8) 进行单步执行，查看右边寄存器值的变化以及下部内存的内容。



三、Cortex-M7 寄存器说明

(1) 寄存器组

Cortex-M 用于数据处理与控制的寄存器组中有 16 个寄存器，其中 13 个（R0 ~ R12）为通用目的寄存器，另外三个具有特殊用途。



MSP 和 PSP 都是 32 位寄存器，但最低两位固定为 0。对于 ARM Cortex-M 处理器，PUSH 和 POP 指令总是 32 位的，栈操作的地址也必须对齐到 32 位的字边界上。一般在用到嵌入式 OS 时才会使用 PSP，此时 OS 内核与应用任务的栈是相互独立的。PSP 的初始值未定义，而 MSP 的初始值则需要在复位流程中从存储器的第一个字中取出。

(2) 程序状态寄存器（xPSR）

程序状态寄存器包括以下三个状态寄存器：

- 应用 PSR（APSR）
- 执行 PSR（EPSR）
- 中断 PSR（IPSR）

三个寄存器各个功能位为：

	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
APSR	N	Z	C	V	Q				GE*							
IPSR												异常编号				
EPSR						ICI/IT	T			ICI/IT						

*GE在Cortex-M4等ARMv7E-M处理器中存在，在Cortex-M3处理器中则不可用。

CSDN @凯之~

位	描　述
N	负标志
Z	零标志
C	进位(或者非借位)标志
V	溢出标志
Q	饱和标志(ARMv6-M 中不存在)
GE[3:0]	大于或等于标志，对应每个字节通路(只存在于 ARMv7E-M，ARMv6-M 或 Cortex-M3 中则不存在)
ICI/IT	中断继续指令(ICI)位，IF-THEN 指令状态位用于条件执行(ARMv6-M 中则不存在)
T	Thumb 状态，总是 1，清除此位会引起错误异常
异常编号	表示处理器正在处理的异常

CSDN @凯之~

这三个寄存器可以通过一个组合寄存器访问，该组合寄存器被称作 xPSR，ARM 汇编器访问 xPSR 时通过“PSR”访问：

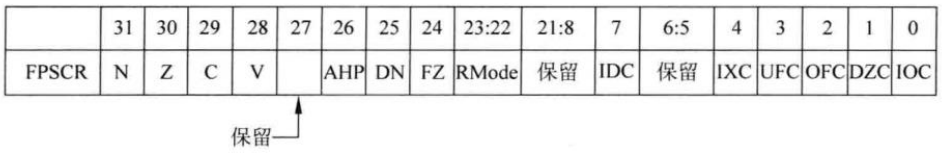
	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
xPSR	N	Z	C	V	Q	ICI/IT	T		GE*	ICI/IT		异常编号				

*GE在Cortex-M4等ARMv7E-M处理器中存在，在Cortex-M3处理器中则不可用。

(3) 浮点寄存器

S0 ~ S31(S)都为 32 位寄存器，可通过浮点指令或利用符号 D0 ~ D15(D 代表双字/双精度)成对访问。 例如，S0 和 S1 成对组成 D0，而 S2 和 S3 则成对组成 D1。

浮点状态和控制寄存器（FPSCR）用于定义浮点运算动作并提供浮点运算结果的状态信息，其位域及各个位的描述如下所示：



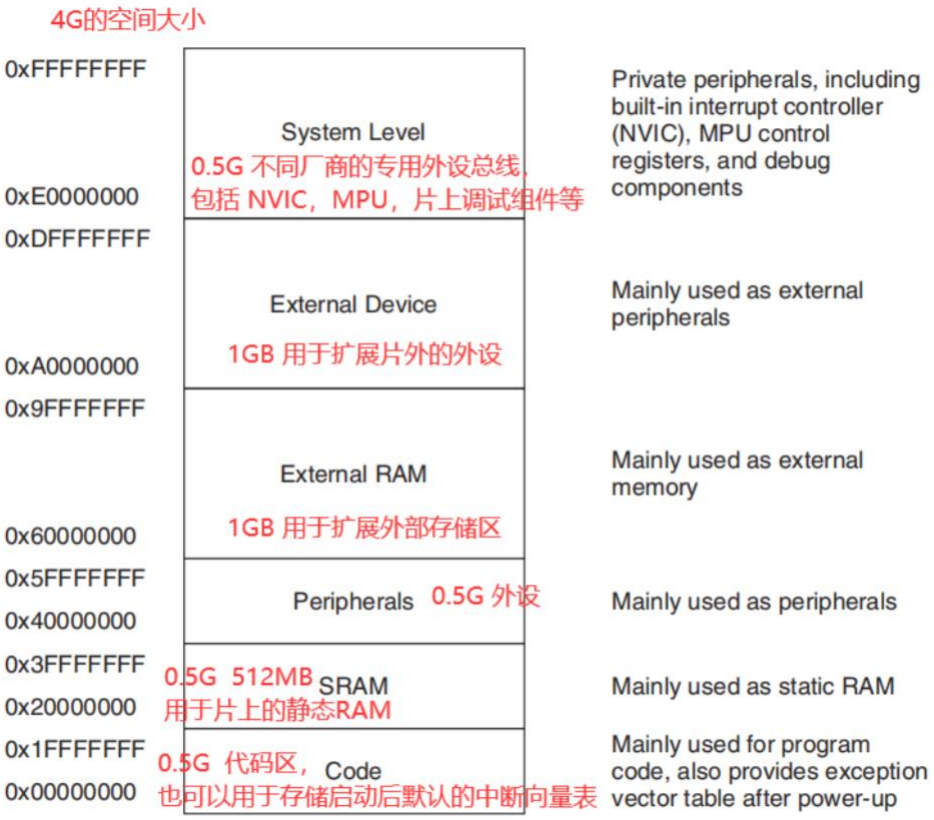
位	描述
N	负标志（由浮点比较运算更新）
Z	零标志（由浮点比较运算更新）
C	进位/借位标志（由浮点比较运算更新）
V	溢出标志（由浮点比较运算更新）
AHP	交替半精度控制位： 0：IEEE半精度格式（默认） 1：交替半精度格式
DN	默认NaN（非数值）模式控制位： 0：NaN操作数会变为浮点运算的输出（默认） 1：任何涉及一个或多个NaN的运算会返回默认的NaN
FZ	清零模式控制位： 0：清零模式禁止（默认）（符合IEE754标准） 1：清零模式使能
RMode	舍入模式控制位,表示的舍入模式基本上适用于所有的浮点指令： 00：最近舍入(RN)模式(默认) 01：正无穷舍入(RP)模式 10：负无穷舍入(RM)模式 11：向零舍入(RZ)模式
IDC	输入非正常累积异常位(结果未在正常数值范围内)
IXC	不精确的累积异常位
UFC	下溢累积异常位
OFC	溢出累积异常位
DZC	被零除累积异常位
IOC	非法操作累积异常位

(4) 中断/异常屏蔽寄存器

名字	功能描述
PRIMASK	1位宽寄存器。置位后关闭NMI（不可屏蔽中断和HardFault外的所有可屏蔽异常/中断。
FAULTMASK	1位宽寄存器。置位后只有NMI可以响应（相比于PRIMASK，可屏蔽的中断/异常多了HardFault）。
BASEPRI	根据优先级屏蔽中断/异常，该寄存器最多有9位（由设计实现的优先级位数决定）。该寄存器定义了被屏蔽优先级的阈值。当它被设置为某个值后，所有优先级号大于等于此值的中断都被关（注意：优先级号越大，优先级越低）；若设置成0，则不关断任何中断。

四、STM32F746G-DISCO 存储架构

(1) 存储器映射



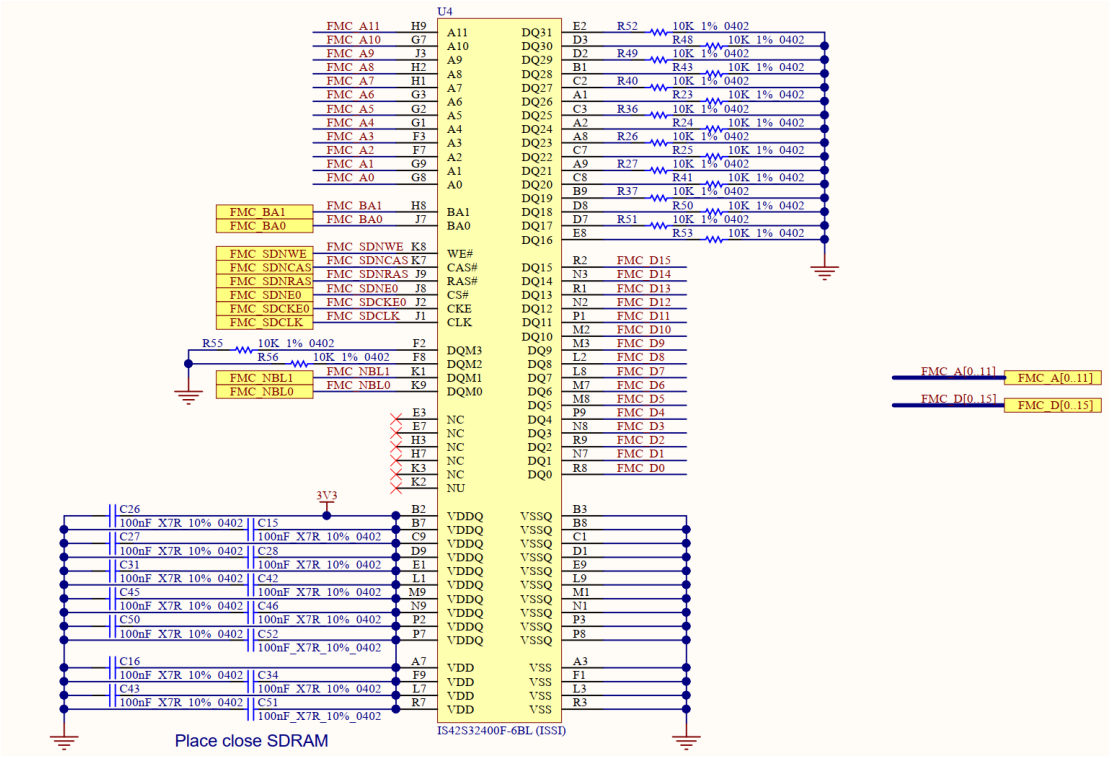
(2) STM32F746NGH6 内部存储

内置 320+16+4K 字节的 SRAM 及 1M 字节的 Flash

```
MEMORY
{
    RAM     (xrw)     : ORIGIN = 0x20000000,   LENGTH = 320K
    FLASH   (rx)      : ORIGIN = 0x8000000,    LENGTH = 1024K
}
```

(3) 外部 SDRAM 内存

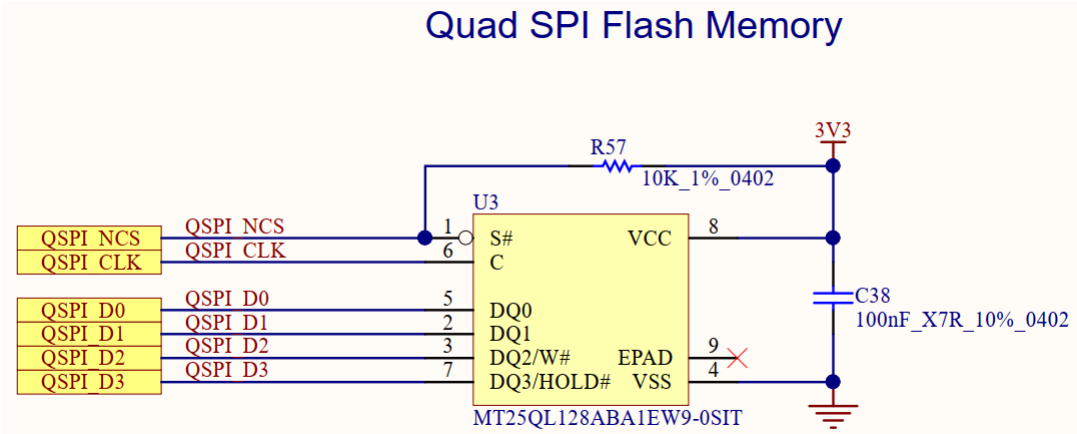
STM32f746G-DISCOV 开发板使用的是 128M 位的 SDRAM（IS42S32400F-6BL）连接到 STM32F746NGH6 的 FMC 接口。只使用了低 16 位的数据(可访问 64M 位)。DQ16 到 DQ31 没有使用，连接一个 10K 欧姆的下拉电阻。



#define BANK_SDRAM_ADDR ((u32)0XC0000000)

(4) 四通道 SPI Nor Flash 闪存

128M 位（16M 字节）的四通道 SPI Nor Flash 闪存(MT25QL128ABA1EW9-0SIT)连接到 STM32F746NGH6 的四通道 SPI 接口。



五、熟悉 ARM 汇编指令

（1） ARM 数据处理指令寻址方式

代码参考

功能：验证 ARM 数据处理指令的寻址方式

```
START:
    MOVS    R0,#0x04                //立即寻址
    MOVS    R1,#0x100
    MOVS    R2,#0x0

    LDR     R0,=0x12345678          //伪指令赋值
    LDR     R2,=( (7<<12)|(7<<8)|(7<<4)|(0<<0) )

    MOV     R5,R2                   //寄存器寻址
    ADD     R7,R1,R2
    CMP     R1,R2

    MOV     R4,R0,ROR #1            //寄存器移位寻址
    ADD     R3,R1,R0,LSL #2
    MOV     R3,R2,LSL #4            //寄存器移位寻址
    SUB     R6,R5,R2,LSL #3         //寄存器移位寻址
    B       START
```

思考

1. LDR R2,=((7<<12)|(7<<8)|(7<<4)|(0<<0))执行后 R2 的值是什么?

（2） ARM 内存访问指令寻址方式

代码参考

功能：验证 ARM 内存访问指令寻址方式

```
START:
        MOV     R1,#0x01
        MOV     R2,#0x04
        MOV     R3,#0x03
        MOV     R4,#0x04
        MOV     R5,#0x05

        LDR     R6,[R2]                 //寄存器间接寻址
        LDR     R6,[R2,#4]
        STR     R3,[R7]
        LDR     R7,[R2,#0x0C]           //基址＋变址寻址

        LDR     R0,=0x100000
        STMIA   R0,{R1-R3}              //多寄存器寻址
        LDR     R0,=0x100000
        STMDB   R0,{R1-R3}              //多寄存器寻址
        LDR     R0,=0x200008
        STMDA   R0,{R1-R3,R5}           //多寄存器寻址
        LDR     R0,=0x210000
        LDMIA   R0,{R1-R5,R7}           //多寄存器寻址
        MOV R0,#0x0
        B       START
```

思考

1. 在内存地址 0x100000 中能否写入 R1-R3 的值？
2. 如果能写入，在内存中的内容是如何分布的？
3. 如果不能写入，该如何修改才能让其写入？

（3） ARM 堆栈指针寄存器

代码参考

功能：验证堆栈指针寄存器的使用特点

```
START:
        MOV     R1,#0x11
        MOV     R2,#0x22
        MOV     R3,#0x33
        MOV     R4,#0x44
        MOV     R5,#0x55
        STMFD   R13!,{R2-R5}            //将寄存器 R2～R5 内容存入堆栈
        MOV     R2,#0X77
        LDMEA   R13!,{R2-R5}
        STMEA   R13!,{R1-R5}            //将寄存器 R1～R5 内容存入堆栈
        STMFD   R13!,{R2-R5}            //将寄存器 R2～R5 内容存入堆栈
        LDMED   R13!,{R2-R5}            //将堆栈内容恢复到寄存器 R2～R5
        MOV     R0,#0
        B       START
```

思考

1. SP 和 R13 是什么关系？
2. LDMED 能否执行成功？如果不成功，该如何修改？
3. 最后 R2-R5 的压栈和出栈数据是否相同？

（4） ARM 程序计数器

代码参考

功能：观察程序计数器的变化，理解ARM指令流水线体系

```
START:
        MOV     R1,#0x04
        MOV     R2,R1
        MOV     R3,R2,LSL #4

        LDR     R3,[R2]
        MOV     R0,PC
        MOV     R3,#0x10
        LDR     R4,[R3,#0x0C]

        LDMIA   R1!,{R4-R7,R12}

        MOV     R2,#0x22
        MOV     R3,#0x33
        MOV     R4,#0x44
        MOV     R5,#0x55

        STMFD   R13!,{R2-R5}
        LDMFD   SP!,{R2-R5}

LOOP:
        BL      ADD_SUB
        B       LOOP

ADD_SUB:
        ADDS    R0,R0,R1
        MOV     PC,LR
```

思考

1. BL 和 B 指令的区别是什么？
2. 在本程序中，LR 的值是多少？在运行的过程中是否发生改变？为什么？

（5） ARM 程序状态寄存器

代码参考

功能：观察数据处理指令执行的结果对程序状态寄存器CPSR位的影响

```
START:
        MOV     R1,#0x04
        MOV     R2,#0x04
        SUBS    R3,R2,R1

        ADDS    R7,R5,R2,LSL #0x4
        MOVS    R6,#8
        CMP     R1,R2

        MOV     R2,R2,ROR #4
        SUB     R6,R5,R2,LSL #3
        B       START
```

思考

1. “SUBS R3,R2,R1”是如何影响标志位和结果的?
2. “CMP R1,R2”是如何影响标志位和结果的?
3. 进位标志“C”，在加法和减法操作中的设置规则是什么?